

卷积神经网络CNN的由来，为什么要用卷积？

原创 还没想好116 于 2019-10-30 09:37:44 发布 阅读量1.2w 收藏 74 点赞数 14

分类专栏: DeepLearning 文章标签: CNN 卷积神经网络

CC 4



脑启社区 文章已被社区收录



DeepLearning 专栏收录该内容

1 篇文章

文章目录

- 卷积神经网络
- 由来
- 加入卷积操作的动机
 - 稀疏连接:
 - 参数共享:
 - 平移不变性
- 池化
- 总结:

卷积神经网络

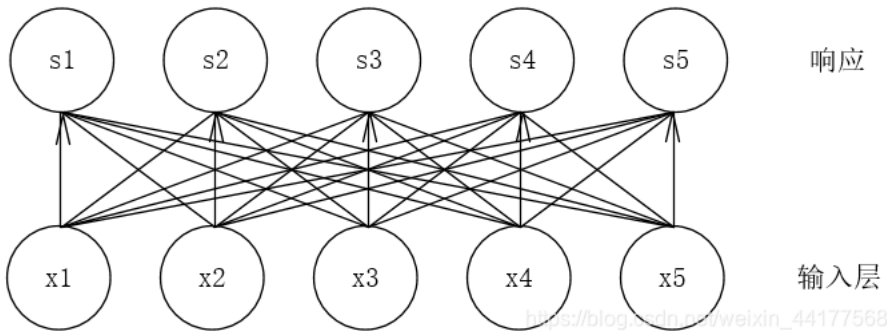
卷积神经网络属于前馈网络的一种，是一种专门处理类似网格数据的神经网络，其特点就是每一层神经元只响应前一层的局部范围内的神经元。

卷积网络一般由：卷积运算+非线性操作（RELU）+池化+若干全连接层。

由来

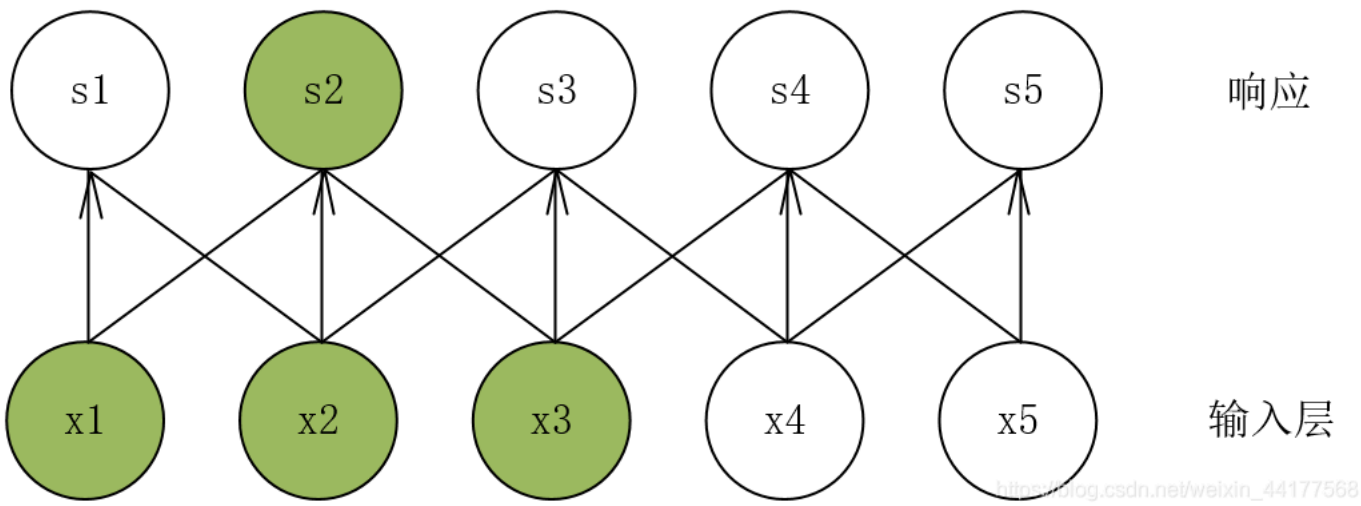
卷积网络之所以叫做卷积网络，是因为这种前馈网络其中采用了卷积的数学操作。在卷积网络之前，一般的网络采用的是矩阵乘法的方式，前一层的都对下一层每一个单元有影响。下面看图理解：

全连接的形式：



图中s是由矩阵乘法产生，响应层的每一个神经元，都会受到输入层每一个神经元的影响，是一种稠密的连接方式。

卷积操作：



可以看出，响应层中的每一个神经元只受到输入层的局部元素的影响。以s2为例，它仅仅受到x1、x2、x3的影响，是一种稀疏的连接方式。

可能到这里，就会有人问，为什么要用这种操作，加入卷积的神经网络意义是什么，或者优势是什么！！！
接着看：

加入卷积操作的动机

传统神经网络都是采用矩阵乘法来建立输入和输出之间的关系，假如我们有 M 个输入和 N个输出，那么在训练过程中，我们需要 M×N 个参数去刻画的关系。当 M 和 N都很大，并且再加几层的卷积网络，这个参数量将会大的离谱。

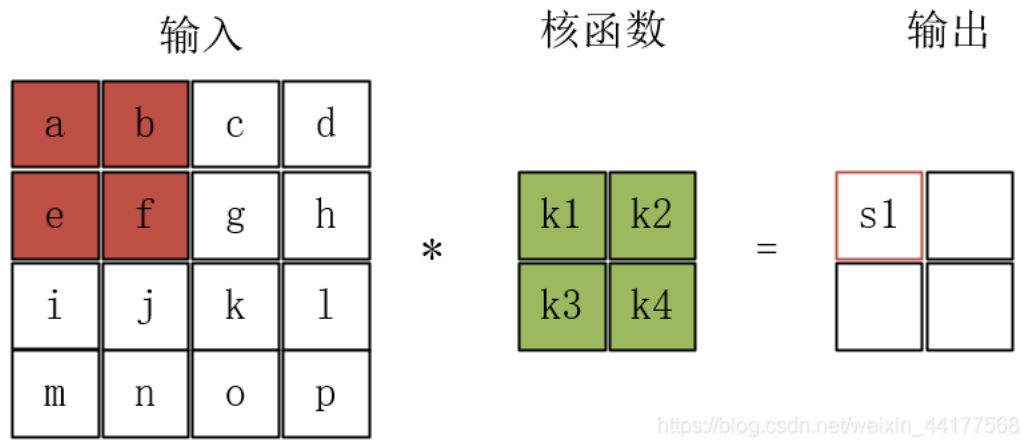
那么对于这种问题，卷积网络怎么应对呢？

卷积运算主要通过三个重要思想来改进上述面来的问题：稀疏连接、参数共享、平移不变性。
——来看

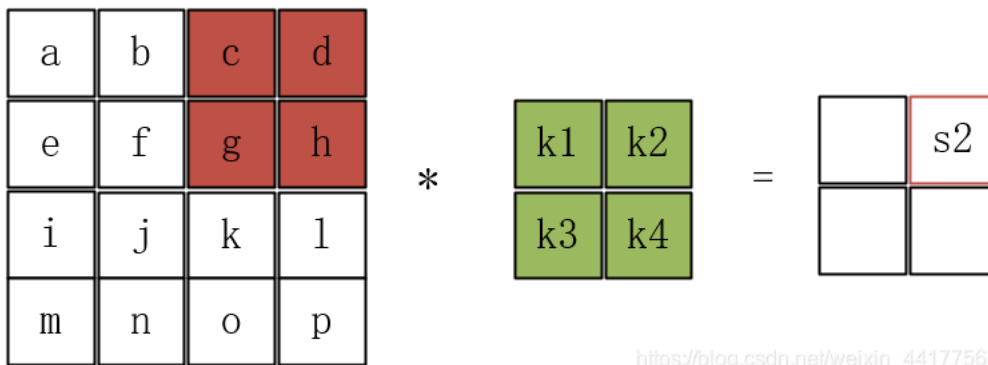
稀疏连接：

上面的图其实就已经可以说明这种稀疏连接的思想。可能有些人不懂这种稀疏连接是怎么实现的？先来说说卷积操作，以一个二维矩阵为输入（可以单通道图片的像素值），卷积产生的稀疏连接根本原因就是这块的核函数，一般的核函数的大小远小于输入的大小。

以下图例：卷积操作可以看做是一种滑窗法，首先，输入维度是4×4，输入中红色部分，先和核函数中的元素对应相乘，就是输出中左上角的元素值 $s = a \times k1 + b \times k2 + e \times k3 + f \times k4$ 。



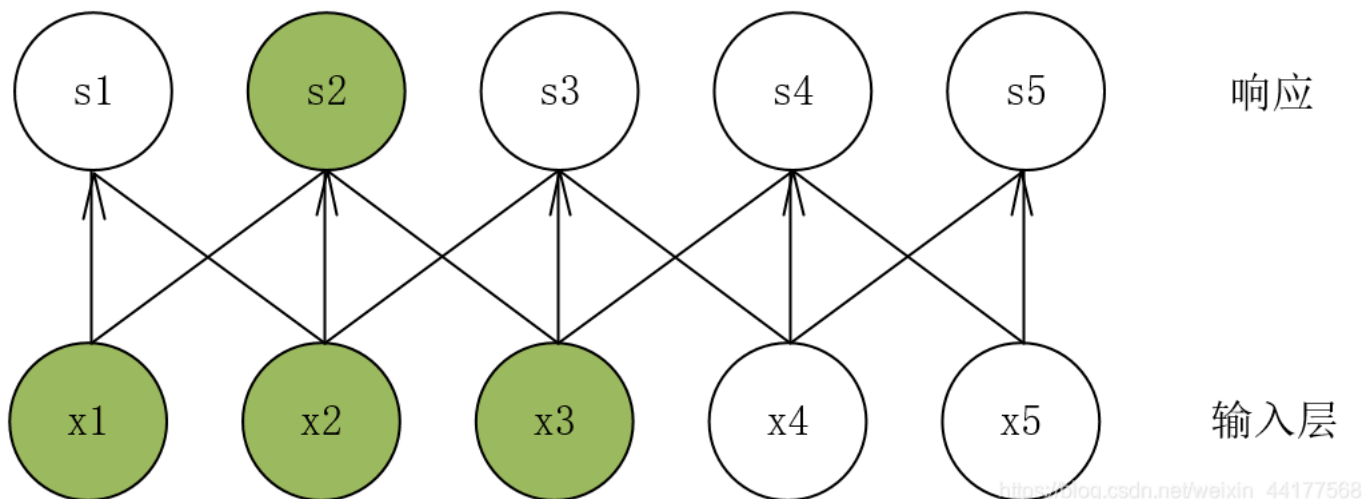
假设步长为2，那么输入中的红色部分将移动到下图的位置，继续卷积运算： $s2 = c \times k1 + d \times k2 + g \times k3 + h \times k4$ 。



https://blog.csdn.net/weixin_44177568

这么一直计算下去就完成了卷积操作，输入4×4的矩阵通过卷积变成2×2的矩阵，其中，输出值s1只和输入中的a、b、e、f有关，和其他元素无关，前面所说的，卷积神经的特点就是每一层神经元只响应前一层的局部范围内的神经元。

继续用这个图来看稀疏连接，s2只和x1、x2、x3有关，感觉这个图看着更明显点！！！！



参数共享：

参数共享是指在一个模型的多个函数中使用相同的参数，它是卷积运算带来的固有属性。

在全连接中，计算每层的输出时，权重矩阵中的元素只作用于某一个输入元素一次；

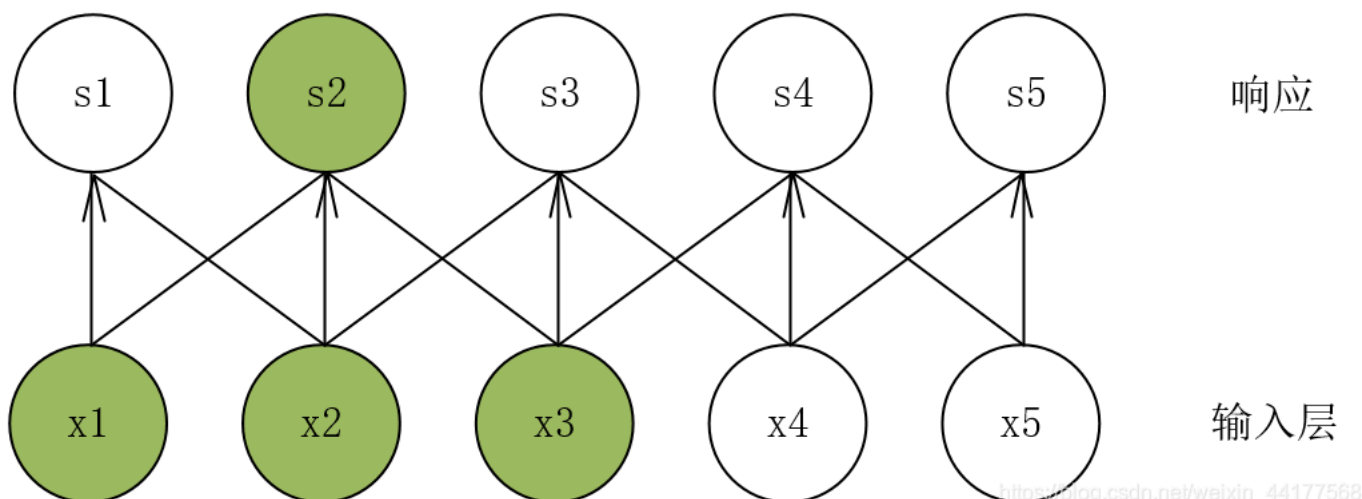
而在卷积神经网络中，卷积核中的每一个元素将作用于每一个局部输入的特定位置上。根据参数共享的思想，我们只需要学习一组参数集合，而不需个位置的每一个参数来进行优化学习，从而大大降低了模型的存储需求。

平移不变性

平移不变性：如果一个函数的输入做了一些改变，那么输出也跟着做出同样的改变，这就叫平移不变性。

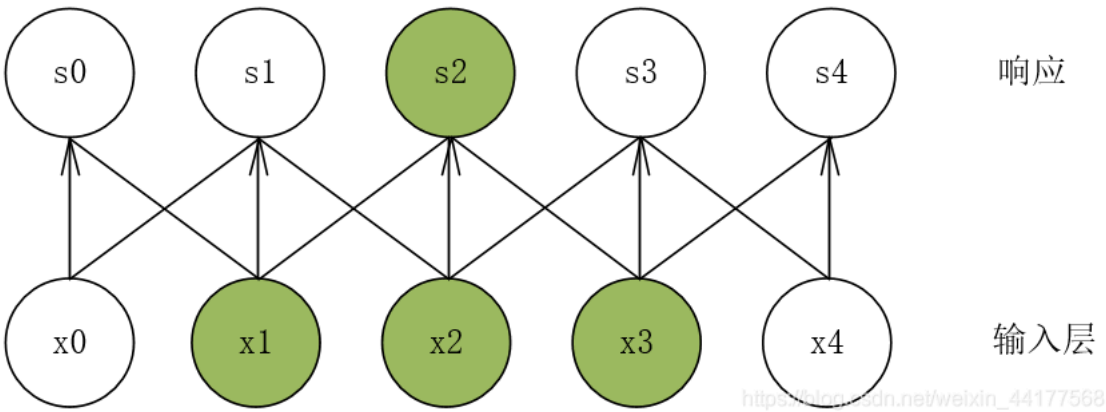
平移不变性是由参数共享的物理意义所得。在计算机视觉中，假如要识别一个图片中是否有一只猫，那么无论这只猫在图片的什么位置，我们都应来，即就是神经网络的输出对于平移不变性来说是等变的。

一下图为例：s2只和x1、x2、x3有关，



https://blog.csdn.net/weixin_44177568

输入层数据向左移动一位：其中响应层的s2也只和x1、x2、x3有关，只是位置发生了稍微的变化，



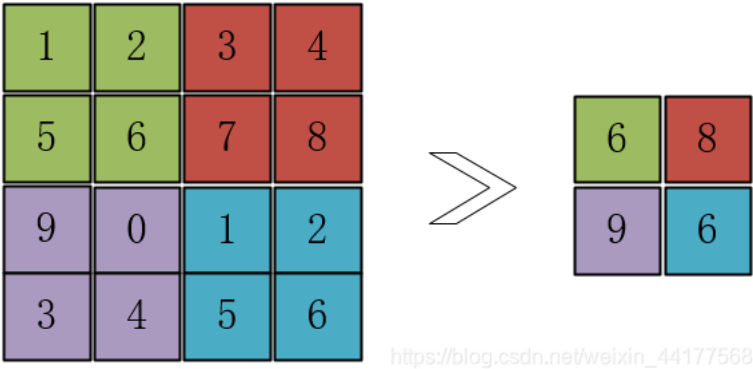
池化

池化：池化函数使用某一位置的相邻输出的总体统计特征来代替网络在该位置的输出。本质是 降采样，可以大幅减少网络的参数量。

常用的池化有：均值池化（mean pooling）、最大池化（max pooling）。

下面用图来形象的说明池化过程：

池化窗口大小为2×2，步长为2，输入矩阵为4×4，窗口开始在输入矩阵的绿色位置，若是最大池化，则取绿色区中的最大值6来表示这个区域，窗口右移最大值为8，以8来代替红色区的值，一次类推，就得到了右边的2×2的经过池化的矩阵，均值池化就是对每个区域求均值。



下面来说说这两种池化的区别与作用：

均值池化：

- 主要用来抑制邻域值之间差别过大，造成的方差过大。
- 如，输入（2,10），通过均值池化后是（6），
- 对于输入的整体信息保存的很好，在计算机视觉中：对背景的保留效果好

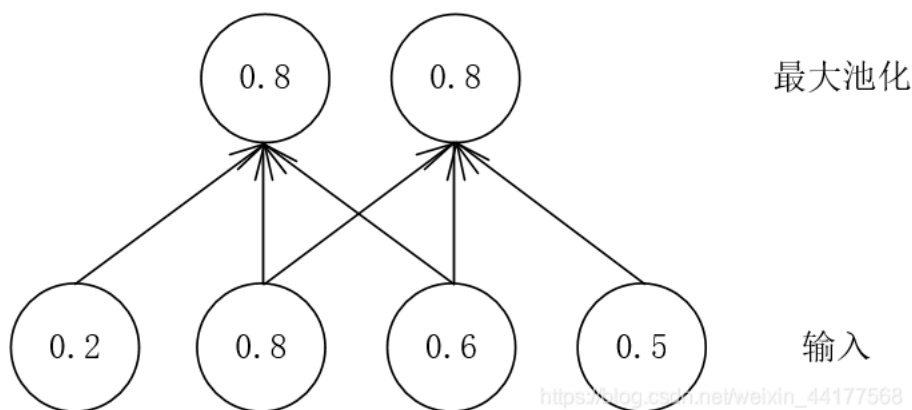
最大池化：

- 能够抑制网络参数误差造成的估计均值偏移的现象。
- 如，输入（1,5,3），最大池化后是（5），假如输入中的参数1，有误差，变为了1.5，这时输入是（1.5,5,3），最大池化后结果还是（5）
- 在计算机视觉中：对纹理的提取较好！

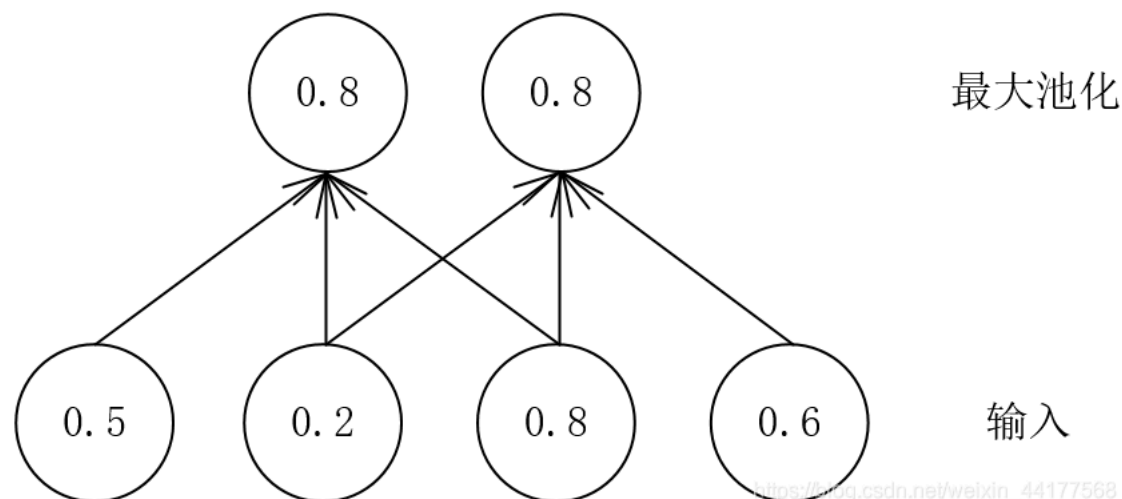
池化操作除了可以明显的减少参数量外，还能够保持对平移、伸缩、旋转等操作的不变性。

以平移不变性为例说明：

下图是输入维度是4×1，池化窗口3×1，步长为1，池化结果如下：



假如输入右移一位，发现最大池化后，结果是一致的。



当然上面只是举个例子，对于实际中的文本或图片数据，池化后肯定是有误差的，但是这点误差换来了参数量的大幅减少，训练速度大幅提升是划算

总结：

卷积操作的参数共享特性使得需要优化的参数数目大幅缩减，提高了训练的速度。并且由于卷积运算主要处理类网格数据，因此对于时间序列以及图析和识别具有显著的优势。

贴一张卷积神经网络在文本分类 中的结构，即TextCNN

